

ME315A LAB02

Arthur Davi Prado de Lima / RA223062

Laboratório 2: Processamento de Bases de Dados em Lote

Questões

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Resposta: Para esse problema, as estatísticas suficientes são o número de vôos atrasados (que satisfazem a condição `ARRIVAL_DELAY > 10`), e o número total de vôos. De forma que, para obter a porcentagem, a operação necessária é: número de vôos atrasados, dividido pelo total de vôos.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a) Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b) Remova observações que tenham valores faltantes em campos de interesse;
 - c) Agrupe o conjunto de dados resultante de acordo com: `dia`, `mês` e `cia. aérea`;
 - d) Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - e) A função deve receber apenas dois argumentos: i. `input`: o conjunto de dados (referente ao lote em questão); ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

`filter`, `lag`

Os seguintes objetos são mascarados por 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
getStats = function(input, pos) {  
  input %>% filter(AIRLINE %in% c("AA", "DL", "UA", "US"), !is.na(ARRIVAL_DELAY),  
    !is.na(DAY), !is.na(MONTH), !is.na(AIRLINE)) %>%  
    group_by(DAY, MONTH, AIRLINE) %>%  
    summarise(  
      total_voos = n(),  
      voos_atrasados = sum(ARRIVAL_DELAY > 10)  
    )  
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
 - a) Configure o tamanho do lote (chunk) para 100 mil registros;
 - b) Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
 - c) Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa)

```
library(readr)  
mycols = cols_only(AIRLINE = 'c', ARRIVAL_DELAY = 'd', DAY = "i", MONTH = "i")  
dados_voos = read_csv_chunked(file = "flights.csv.zip",  
  callback = DataFrameCallback$new(getStats),  
  chunk_size = 1e5,  
  col_types = mycols)
```

```
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.  
`summarise()` has regrouped the output.
```

[illegible]

```
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
i Summaries were computed grouped by DAY, MONTH, and AIRLINE.
i Output is grouped by DAY and MONTH.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(DAY, MONTH, AIRLINE))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

4. Crie uma função chamada `computeStats` que:

- a) Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b) Retorne as informações em um tibble contendo apenas as seguintes colunas:
 - i. Cia: sigla da companhia aérea;
 - ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1].

```
computeStats = function(stats) {
  stats %>%
  group_by(DAY, MONTH, AIRLINE) %>%
  summarise(Perc = sum(voos_atrasados) / sum(total_voos)) %>%
  mutate(Data = as.Date(paste("2015", MONTH, DAY, sep = "-"))) %>%
  select(Cia = AIRLINE, Data, Perc)
}
tabela_final <- computeStats(dados_voos)
```

```
`summarise()` has regrouped the output.
Adding missing grouping variables: `DAY`, `MONTH`
i Summaries were computed grouped by DAY, MONTH, and AIRLINE.
i Output is grouped by DAY and MONTH.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(DAY, MONTH, AIRLINE))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- a) Instale e carregue os pacotes `ggcal` e `ggplot2`.
- b) Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c) Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 - i. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x,y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - iii. Retornar para o usuário a base do calendário criada acima.
- d) Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e) Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
# install.packages("devtools")
# devtools::install_github("jayjacobs/ggcal")
# install.packages("ggplot2")
library(ggcal)
library(ggplot2)

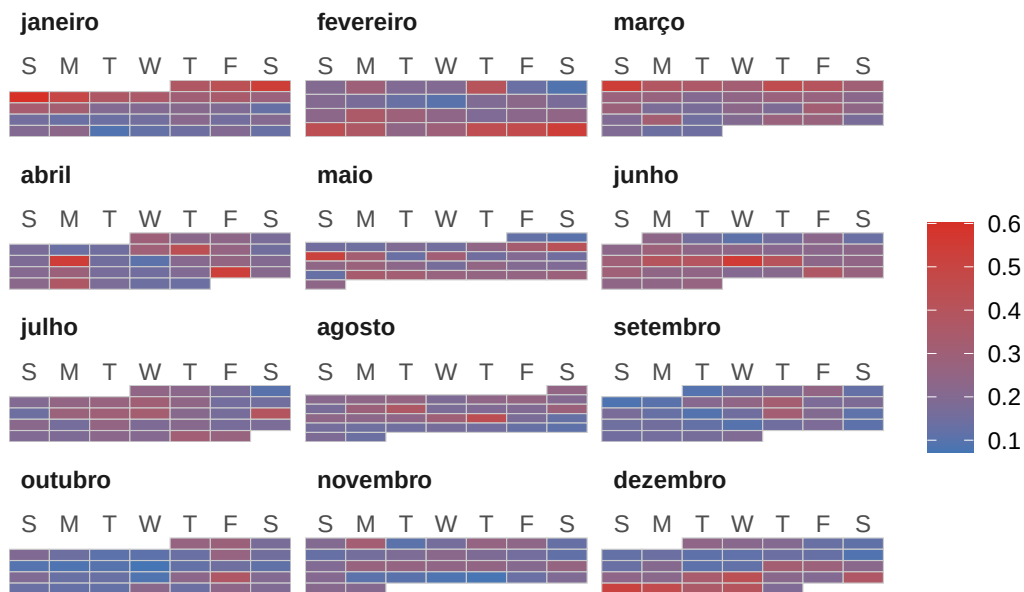
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

baseCalendario <- function(stats, cia) {
  dados_filtrados <- stats %>% filter(Cia == cia)
  ggcal(dados_filtrados$Data, dados_filtrados$Perc)
}

cAA <- baseCalendario(tabela_final, "AA")
cDL <- baseCalendario(tabela_final, "DL")
cUA <- baseCalendario(tabela_final, "UA")
cUS <- baseCalendario(tabela_final, "US")

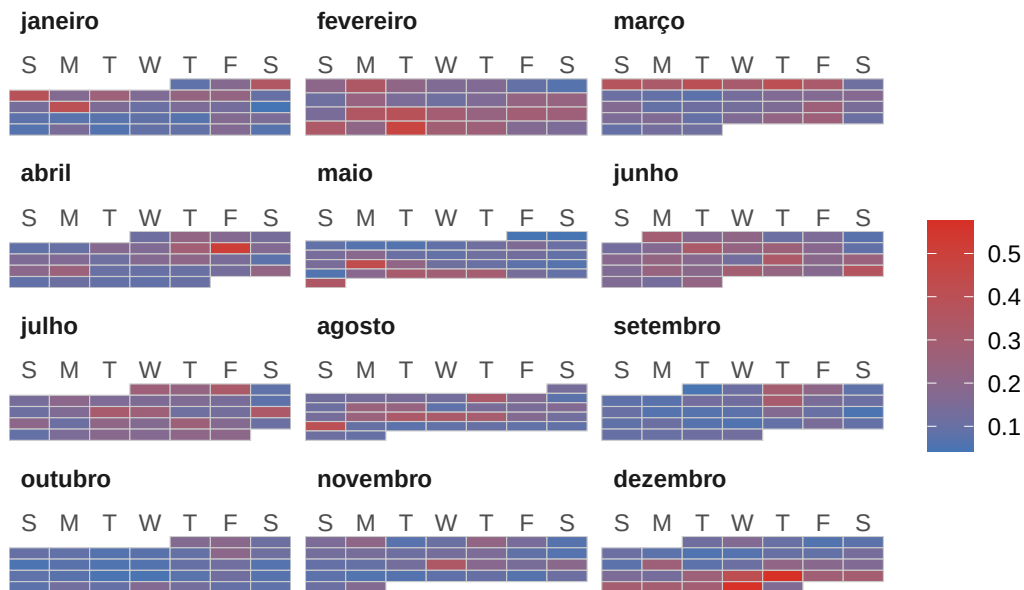
cAA + pal + ggtitle("Percentual de Atrasos - AA")
```

Percentual de Atrasos – AA



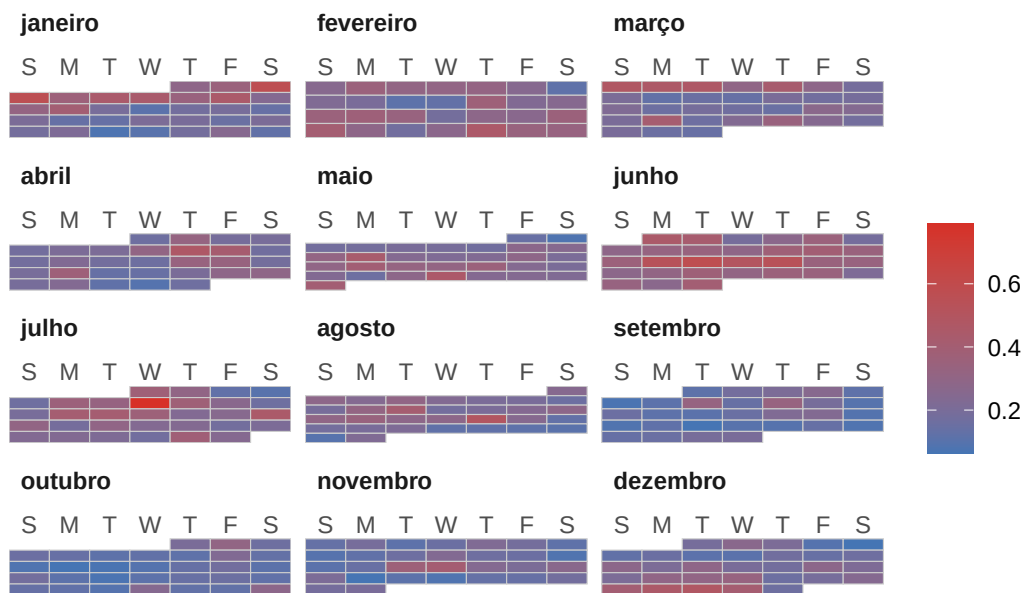
```
cDL + pal + ggtitle("Percentual de Atrasos - DL")
```

Percentual de Atrasos – DL



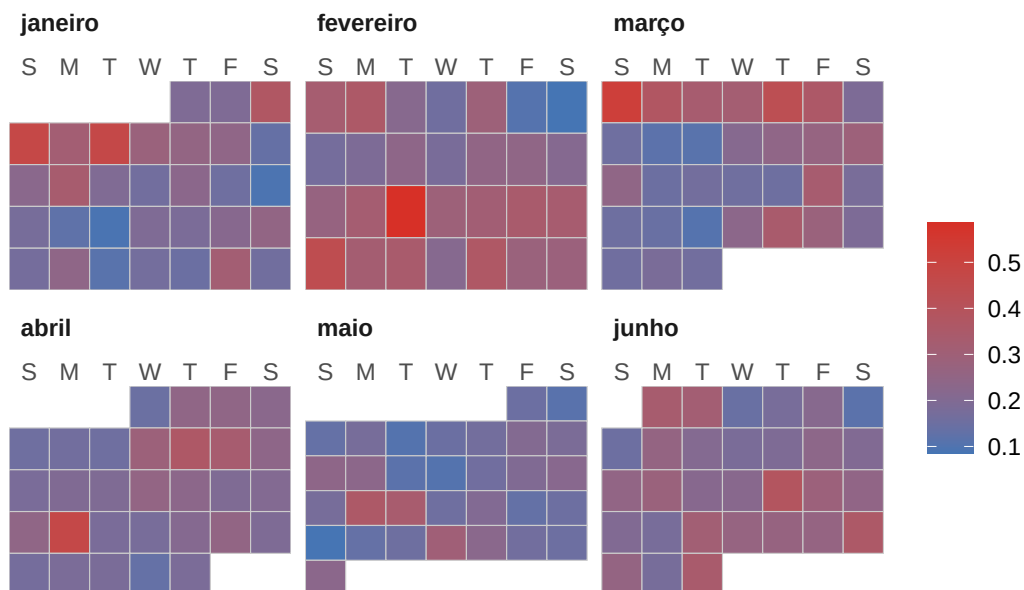
```
cUA + pal + ggtitle("Percentual de Atrasos - UA")
```

Percentual de Atrasos – UA



```
cUS + pal + ggtitle("Percentual de Atrasos - US")
```

Percentual de Atrasos – US



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 21:40:10 -03"
```